C'est bien noté ! C'est une excellente précision par rapport au standard CloudEvents :

1. **Le payload (data)** reste une boîte noire (une simple chaîne de caractères) pour l'infrastructure. C'est aux consommateurs finaux de le parser s'ils en ont besoin.
2. **Les extensions métiers** (qui sont à la racine de l'événement CloudEvent, au même niveau que id ou source) doivent être extraites pour permettre à Starburst de faire des filtres rapides dessus (WHERE ext_tenant = 'FR').

Voici la mise à jour complète (la requête Starburst ET le code Spark) pour intégrer ces éléments.

1. La table Starburst (Mise à jour)

Il faut ajouter tes extensions métiers comme de nouvelles colonnes. Dans l'exemple ci-dessous, j'ai ajouté ext_tenant_id et ext_correlation_id, que tu devras remplacer par tes vrais noms d'extensions.

SQL

```
CREATE TABLE dh_poc_ice.pocspark (
  specversion VARCHAR,
  type VARCHAR,
  source VARCHAR,
  subject VARCHAR,
  id VARCHAR,
  time TIMESTAMP(6) WITH TIME ZONE,
  datacontenttype VARCHAR,
  data VARCHAR, -- Le payload métier reste une simple chaîne JSON non parsée
  -- Extensions métiers (à adapter avec tes vrais champs)
  ext_tenant_id VARCHAR,
  ext_correlation_id VARCHAR,
  -- Colonne technique
  dh_poc_insert_timestamp TIMESTAMP(6) WITH TIME ZONE
)
WITH (
  format = 'PARQUET',
```

```
  location = 's3a://bu002i004226/pocspark'
);
```

2. Le code PySpark (Mise à jour)

On met à jour le schéma statique pour qu'il "attrape" les extensions métiers à la volée tout en ignorant ce qu'il y a à l'intérieur de data.

Python

```
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType, TimestampType
from pyspark.sql.functions import current_timestamp, to_json, col

spark = SparkSession.builder \
  .appName("poc-streaming-iceberg") \
  .config("spark.eventLog.enabled", "false") \
  .config("spark.sql.extensions",
"org.apache.iceberg.spark.extensions.IcebergSparkSessionExtensions") \
  .config("spark.sql.catalog.dh_poc_ice", "org.apache.iceberg.spark.SparkCatalog") \
  .config("spark.sql.catalog.dh_poc_ice.type", "hadoop") \
  .config("spark.sql.catalog.dh_poc_ice.warehouse", "s3a://bu002i004226/") \
  .getOrCreate()

spark.sparkContext.setLogLevel("WARN")

# 1. Mise à jour du Schéma (Ajout des extensions)
event_schema = StructType([
  StructField("specversion", StringType(), True),
  StructField("type", StringType(), True),
  StructField("source", StringType(), True),
  StructField("subject", StringType(), True),
  StructField("id", StringType(), True),
```

```
    StructField("time", TimestampType(), True),
    StructField("datacontenttype", StringType(), True),
    # On force la lecture de 'data' comme une chaîne de caractères
    StructField("data", StringType(), True),
    # --- Extensions Métiers ---
    StructField("ext_tenant_id", StringType(), True),
    StructField("ext_correlation_id", StringType(), True)
])

# 2. Lecture
df_raw = spark.readStream \
    .format("json") \
    .schema(event_schema) \
    .load("s3a://bu002i004226/poc_streaming/input_raw/")

# 3. Ajout du timestamp d'insertion
df_processed = df_raw.withColumn("dh_poc_insert_timestamp", current_timestamp())

# 4. Écriture dans Iceberg
query = df_processed.writeStream \
    .format("iceberg") \
    .outputMode("append") \
    .trigger(processingTime="30 seconds") \
    .option("checkpointLocation",
"s3a://bu002i004226/poc_streaming/checkpoint_iceberg/") \
    .toTable("dh_poc_ice.pocspark")

query.awaitTermination()
```

⚠️ Un détail technique critique sur le champ data avec Spark

Si le système qui t'envoie les données écrit le JSON de cette façon (avec data comme un **objet** imbriqué) :

JSON

```
{
  "id": "123",
  "data": {"nom": "test", "valeur": 42}
}
```

Le lecteur JSON de Spark va renvoyer null pour la colonne data car il s'attend à trouver une simple chaîne de texte (puisqu'on a mis StringType()), et non un sous-objet.

La parade :

Si tu constates que ta colonne data est vide dans Starburst à cause de ça, il faudra lire le fichier ligne par ligne (en format text) et utiliser la fonction get_json_object() de Spark pour extraire les champs sans les parser. Mais si ton système source envoie le payload "échappé" (ex: "data": "{\"nom\": \"test\"}"), alors le code ci-dessus fonctionnera parfaitement du premier coup !

C'est une excellente approche pour valider ton pipeline de bout en bout. Pour répondre à tes besoins, nous allons séparer le travail en deux parties :

1. **Le Producteur (Mock Generator)** : Un petit script Python qui va simuler l'arrivée des fichiers dans ton bucket "cloudevent" toutes les 2 secondes avec des données aléatoires (UIDs, dates, etc.).
2. **Le Consommateur (Spark Streaming)** : Le code Spark mis à jour qui va lire ces fichiers, traiter le champ data comme du texte brut (JSON blob) et insérer le tout dans ta table Starburst/Iceberg.

---

1. Le Générateur de données (Mock Python Script)

Ce script va créer des fichiers JSON conformes au standard CloudEvent dans ton bucket d'entrée.

Python

```
import uuid

import json

import time

from datetime import datetime

import boto3 # Assure-toi d'avoir boto3 installé pour écrire sur COS


# Configuration

BUCKET_NAME = "bu002i004226"

PREFIX = "poc_streaming/input_raw/"

MAX_ITERATIONS = 20  # Nombre max de fichiers à générer

SLEEP_TIME = 2      # 2 secondes entre chaque fichier


# Initialisation client S3 (IBM COS)

s3 = boto3.client('s3',

    endpoint_url='https://s3.direct.eu-fr2.cloud-object-storage.appdomain.cloud'

)
```

```
def generate_mock_event(i):
    event = {
        "specversion": "1.0",
        "type": "com.enterprise.iot.sensor.data",
        "source": "/sensors/thermal-01",
        "subject": "telemetry",
        "id": str(uuid.uuid4()),
        "time": datetime.utcnow().isoformat() + "Z",
        "datacontenttype": "application/json",
        # Le contenu métier "data" est un objet JSON ici
        "data": {
            "temp": 20 + i,
            "status": "OK",
            "battery_level": 0.95 - (i * 0.01)
        },
        # Extensions métiers
        "ext_tenant_id": "FR_TEAM_01",
        "ext_correlation_id": f"CORR-{i:04d}"
    }
    return event

print(f"--- Démarrage de la génération de {MAX_ITERATIONS} événements ---")

for i in range(MAX_ITERATIONS):
    event_data = generate_mock_event(i)
    file_name = f"{PREFIX}event_{i}_{event_data['id'][:8]}.json"

    # Upload sur COS
```

```
    s3.put_object(
        Bucket=BUCKET_NAME,
        Key=file_name,
        Body=json.dumps(event_data)
    )

    print(f"[{i+1}/{MAX_ITERATIONS}] Fichier uploadé : {file_name}")
    time.sleep(SLEEP_TIME)

print("--- Génération terminée ---")
```

---

2. Le Code Spark Streaming (Traitement Iceberg)

Voici la version de streaming_app.py qui récupère ces fichiers et force le stockage du champ data en texte JSON sans le parser.

Python

```
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType, TimestampType, MapType
from pyspark.sql.functions import current_timestamp, to_json, col

spark = SparkSession.builder \
    .appName("poc-streaming-final") \
    .config("spark.eventLog.enabled", "false") \
    .config("spark.sql.extensions", "org.apache.iceberg.spark.extensions.IcebergSparkSessionExtensions") \
    .config("spark.sql.catalog.dh_poc_ice", "org.apache.iceberg.spark.SparkCatalog") \
    .config("spark.sql.catalog.dh_poc_ice.type", "hadoop") \
    .config("spark.sql.catalog.dh_poc_ice.warehouse", "s3a://bu002i004226/pocspark/") \
    .getOrCreate()
```

```
# Schéma CloudEvent (Note : 'data' est lu comme une Map ou un Struct pour être capturé)
event_schema = StructType([
    StructField("specversion", StringType(), True),
    StructField("type", StringType(), True),
    StructField("source", StringType(), True),
    StructField("subject", StringType(), True),
    StructField("id", StringType(), True),
    StructField("time", TimestampType(), True),
    StructField("datacontenttype", StringType(), True),
    # On lit 'data' comme une Map flexible pour ne pas perdre d'info
    StructField("data", MapType(StringType(), StringType()), True),
    StructField("ext_tenant_id", StringType(), True),
    StructField("ext_correlation_id", StringType(), True)
])

# 1. Lecture du bucket
df_raw = spark.readStream \
    .format("json") \
    .schema(event_schema) \
    .option("maxFilesPerTrigger", 1) \
    .load("s3a://bu002i004226/poc_streaming/input_raw/")

# 2. Transformation
# On transforme la Map 'data' en chaîne de caractères JSON brute (String)
# Et on ajoute le timestamp technique
df_final = df_raw \
```

```
    .withColumn("data", to_json(col("data"))) \
    .withColumn("dh_poc_insert_timestamp", current_timestamp())

# 3. Écriture vers Starburst/Iceberg
# Le trigger à 2 secondes correspond à ton besoin de réactivité
query = df_final.writeStream \
    .format("iceberg") \
    .outputMode("append") \
    .trigger(processingTime="2 seconds") \
    .option("checkpointLocation",
"s3a://bu002i004226/poc_streaming/checkpoint_iceberg_final/") \
    .toTable("dh_poc_ice.pocspark")

query.awaitTermination()
```

Pourquoi c'est la solution idéale pour ton test :

1. **Donnée brute préservée** : En utilisant to_json(col("data")), même si ton CSV ou JSON d'entrée contient des objets complexes, Spark les transforme en une seule "chaîne de caractères" JSON dans ta colonne Starburst. C'est exactement ce qui est demandé pour ne pas parser le contenu métier.
2. **Synchronisation** : Si tu lances le script de génération en même temps que le job Spark, tu verras en temps réel (toutes les 2 secondes) les lignes s'ajouter dans ta table.
3. **UIDs et Timestamps** : Le générateur utilise uuid4() pour garantir que chaque événement est unique, simulant une vraie source de données.
4. **Max Iterations** : Le script Python s'arrête tout seul après 20 fichiers, ce qui évite de consommer ton stockage S3 inutilement si tu oublies de l'éteindre.

Pour tester, lance d'abord le job Spark, puis lance le script de génération. Tu pourras ensuite aller dans Starburst et faire un SELECT * FROM dh_poc_ice.pocspark ORDER BY dh_poc_insert_timestamp DESC; pour voir tes données arriver !